

2025年第十八届“认证杯”数学中国

数学建模网络挑战赛第一阶段论文

题 目 无人机配送路径优化：模拟退火算法的应用研究
关 键 词 模拟退火算法、旅行商问题(TSP)、启发式算法

摘 要：

随着无人机技术的不断发展与成熟，无人机配送逐渐成为新热点。本文对无人机配送路径进行优化研究，探讨在存在禁飞区的情况下，如何实现多个无人机分区配送的路径优化。

针对问题一，单个无人机需一次配送 35 个需求点。通过建立以路径最小化为目标函数的模型，并运用模拟退火算法求解最优路径。最终得出最短路径为 [13, 10, 14, 15, 16, 17, 21, 20, 19, 18, 11, 8, 3, 1, 2, 5, 4, 6, 7, 9, 12, 26, 27, 28, 29, 31, 30, 34, 36, 35, 33, 32, 25, 24, 23, 22, 13]，其最短路径长度为 470 个单位。

针对问题二，属于多无人机配送问题。我们依据以出发点为圆心的扇形分区需求点密度分析，将城市划分为五个区域，其中稀疏区为 2、4，稠密区为 1、3、5。在每架无人机携带相同包裹的前提下，稀疏区安排一架无人机，稠密区安排两架无人机进行配送。

针对问题三，对于问题一中的单无人机问题，我们定义一个燃料消耗函数，将该问题转化为带载重变化的旅行商问题（TSP）。这需要动态计算燃料消耗，可以采用动态规划或启发式算法，优先访问远距离、大载重的目的地。对于问题二中的多无人机问题，为每个区域建立目标函数，即总燃料消耗等于各无人机路径的燃料消耗之和，也等于各无人机的载重之和。我们仍然可以使用模拟退火算法进行求解，最终进行整合。

针对问题四，在禁飞区的约束条件下，可沿禁飞区边缘将城市划分为上下半区。在上半区，以 13 号点为起点，28 号点为终点；在下半区，以 28 号点为起点，沿边缘飞至 7 号点，最后从 12 号点返回 13 号点。该建模是基于无人机无法准确感知禁飞区边缘的假设。分别对上下半区运用模拟退火算法进行路径优化。

综上所述，本文基于模拟退火算法，解决了无人机在不同约束条件下的配送路径优化问题。为单架和多架无人机的路径规划提供了有效的解决方案。

参赛队号：#

所选题目：D 题

参赛密码_____

（由组委会填写）

英文摘要

With the continuous development and maturity of UAV technology, UAV distribution has gradually become a new hot spot. This paper carries out optimization research on drone distribution paths, and explores how to achieve path optimization of multiple drone partition distribution in the presence of no-fly zones.

For problem 1, a single UAV needs to deliver 35 demand points at a time. The optimal path is solved by building a model with **path minimization** as the objective function and applying **simulated annealing algorithm**. The final shortest path is [13, 10, 14, 15, 16, 17, 21, 20, 19, 18, 11, 8, 3, 1, 2, 5, 4, 6, 7, 9, 12, 26, 27, 28, 29, 31, 30, 34, 36, 35, 33, 32, 25, 24, 23, 22, 13], and its shortest path length is 470 units.

For problem 2, it belongs to the problem of multi-drone distribution. We divide the city into five zones based on the density analysis of the demand points in the fan-shaped zoning with the departure point as the center of the circle, in which the sparse zones are **2 and 4**, and the dense zones are **1, 3, and 5**. Under the premise that each drone carries the same parcel, **one drone** is arranged in the sparse zone, and **two drones** are arranged in the dense zone to carry out the delivery.

For Problem 3, for the single drone problem in Problem 1, we define a fuel consumption function. Thus the problem is transformed into a **Traveler's Problem with Load Variation (TSP)**. Fuel consumption needs to be computed dynamically, and **dynamic planning** or **heuristic algorithms** can be used to prioritize visits to long-distance destinations with large loads. For the multi-UAV problem in Problem 2, an objective function is established for each region, i.e., the total fuel consumption is equal to the sum of the fuel consumption of the paths of the UAVs, and also to the sum of the loads of the UAVs, which can still be solved by us using **simulated annealing algorithms** for the final integration.

For Problem 4, the city can be divided into upper and lower half zones along the edge of the no-fly zone, subject to the constraints of the no-fly zone. **In the upper half of the zone, point 13 is the starting point and point 28 is the end point; in the lower half of the zone, point 28 is the starting point, and the UAV flies along the edge to point 7, and finally returns to point 13 from point 12.** The modeling is based on the assumption that UAVs cannot accurately perceive the edge of the no-fly zone. A simulated annealing algorithm is applied to the upper and lower half of the zone for path optimization, respectively.

In summary, this paper solves the distribution path optimization problem for UAVs under different constraints based on simulated annealing algorithm. An effective solution is provided for path planning of single and multiple UAVs.

1 问题重述

1.1 问题背景

随着物流行业的快速发展，无人机技术逐渐成为短途小件货物配送的新趋势。许多城市开始探索利用无人机进行快递和外卖配送，以提高配送效率、降低成本并减少交通拥堵。无人机能够快速穿梭于城市街区，实现点对点的直接配送，尤其适用于城市中心等交通繁忙区域。

然而，无人机配送也面临着诸多挑战。一方面，城市环境复杂多变，可能存在禁飞区、高楼大厦、气象条件不稳定等因素，这些都会对无人机的飞行安全和配送效率产生影响。另一方面，无人机的载重能力有限，如何合理分配包裹、优化飞行路径以减少燃料消耗和时间成本，成为亟待解决的问题。

1.2 问题提出

根据题目要求，需建立数学模型，解决以下问题：

对于问题一，忽略各种因素，同时忽略无人机的载重量对燃料消耗的影响，目标是使一架无人机完成所有配送任务并返回出发点的总飞行距离最短。

对于问题二，有多架无人机可供使用考虑航程，以最小化总燃料消耗完成递送任务。该问题可以简化为最小化所有无人机的总飞行距离。

对于问题三，基于问题一和问题二，重新建模，考虑无人机载重对燃料消耗的影响，优化无人机目的地列表和包裹分配，以减少总燃料消耗。

对于问题四，城市中存在禁飞区，需为单架无人机安排目的地列表，使其在避开禁飞区的前提下，完成所有配送任务并返回出发点的总飞行距离最短。

1.3 本文工作

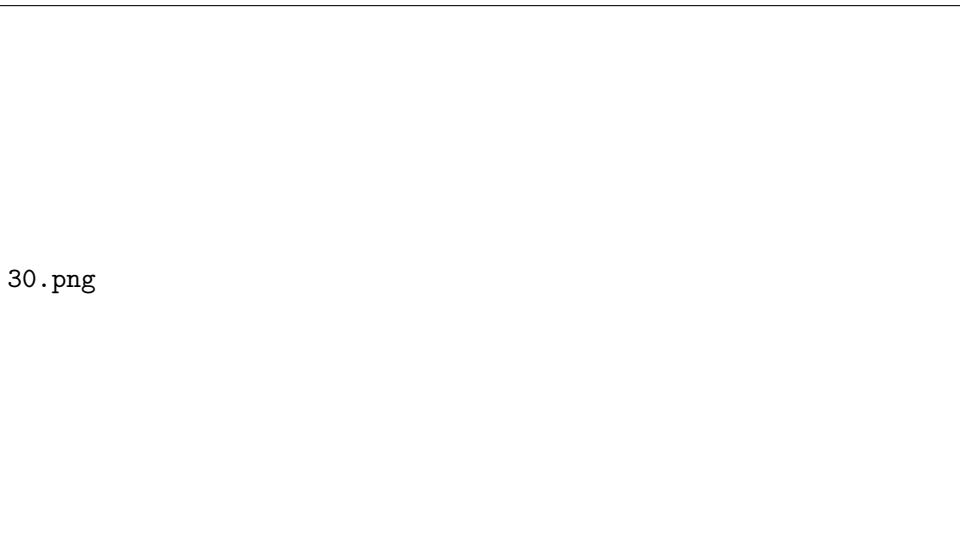


图 1: 本文思维导图

2 问题分析

2.1 问题一分析

这是一个典型的旅行商问题（TSP）变种。无人机需从仓库出发，访问所有分配的目的地后返回仓库，目标是寻找最短路径，即实现最少燃料消耗。将仓库和目的地视为图中的节点，节点间的距离采用欧几里得距离（或实际道路距离），目的是最小化总飞行距离，即路径长度。

考虑到旅行商问题的求解算法可分为精确算法和启发式算法。精确算法如动态规划（Held-Karp 算法，仅适用于小规模问题）；启发式算法包括贪心算法（最近邻法）、模拟退火、遗传算法等（适用于大规模问题）。由于图中有 36 个节点，属于大规模问题，因此选用启发式算法中的模拟退火算法来寻找一个有序的目的地列表。

2.2 问题二分析

这是多旅行商问题（MTSP）或车辆路径问题（VRP）的简化版。在满足每架无人机装载量大致相等的条件下，目标是使所有无人机完成任务的总燃料消耗最小化。由于暂不考虑载重量对燃料消耗的影响，可以简化为最小化所有无人机的总飞行距离。

将仓库和目的地建模为图，目标是将目的地划分为若干子集，每个子集由一架无人机服务，各子集大小（负载）大致相等，同时使所有无人机路径长度之和最小。可以先运用聚类算法（如 K-means）将目的地按地理位置分组，每组对应一架无人机，然后对每组运用 TSP 算法中的启发式算法来规划路径。最终输出每架无人机的包裹分配列表以及每架无人机的飞行路径。

2.3 问题三分析

对于问题一中的单无人机问题，燃料消耗与载重和距离相关，假设燃料消耗率等于飞机的载重乘以飞行距离。因此，可以定义燃料消耗函数。当 F 值最小时，路径规划最为合理。由于每次卸货后载重减少，路径顺序对总燃料消耗有直接影响。该问题转化为带载重变化的 TSP，需要动态计算燃料消耗。可以采用动态规划或启发式算法，优先访问远距离、大载重的目的地。

对于问题二中的多无人机问题，在分配包裹时需同时考虑路径和载重。在问题二的基础上，为每个区域建立目标函数：总燃料消耗 = 各无人机路径的燃料消耗的总和 = 各无人机的载重乘以飞行距离的总和，最后进行整合。

2.4 问题四分析

该问题是城市中存在障碍物约束的 TSP，路径需避开禁飞区。在满足禁飞区约束的条件下，目标是最小化无人机完成所有配送任务并返回出发点的总飞行距离。

在计算节点之间的距离时，不能简单地使用直线距离，而需要考虑禁飞区的存在。可以采用网格化方法，将城市地图划分为一个个小网格，将禁飞区内的网格标记为不

可通过。然后运用路径搜索算法，从一个节点到另一个节点搜索最短路径，避开禁飞区。通过这种方式计算出所有节点之间的最短路径距离矩阵，再将其作为 TSP 问题的输入，运用问题一中提到的求解方法求解。

此外，对于禁飞区这种障碍物，还可以采用虚拟节点的方法。将禁飞区的边界视为障碍物，将无人机的飞行路径限制在障碍物的外侧。在路径规划时，将障碍物边界上的点作为虚拟节点，无人机只能通过这些虚拟节点绕过障碍物。

3 模型假设

为简化问题，作如下假设：

- 1、飞行速度假设：无人机的飞行速度在一般情况下保持恒定不变。
- 2、载重量与燃料假设：无人机的载重量对燃料消耗的影响呈线性关系，即载重量增加时，燃料消耗相应增加。
- 3、飞行路径假设：无人机的飞行路径为直线。
- 4、信息实时更新假设：无人机可以实时获取包裹的收货地址和状态信息，确保任务分配的准确性和及时性。
- 5、无人机航程充足假设：每架无人机的航程足够完成分配的任务列表，无需中途返回充电或更换电池。
- 6、飞行障碍假设：无人机在飞行过程中不受其他交通工具的影响。

4 符号系统

表1：无人机送货模型变量表

符号	说明
n	节点数量（包裹数量）
D_{ij}	节点 i 到节点 j 的距离
i, j	节点索引，取值范围为 $1, 2, \dots, n$
C	无人机最大飞行距离
W	无人机最大载重量
T	无人机最大飞行时间
w_i	节点 i 处包裹的重量
t_i	在节点 i 处的作业时间
F	燃料消耗总量
k	每个包裹的重量

注：未申明的变量以其在符号出现处的具体说明为准。

5 模型建立

5.1 问题一模型建立与求解

将题目中给出的图 1 放入 Auto CAD 中，以图片左下角为坐标原点建立直角坐标系，测得各点坐标，如图 2 所示。接着通过 R 语言绘制出空间分布图，如图 3 所示，其中红点 13 代表仓库，蓝点为各个目的地。

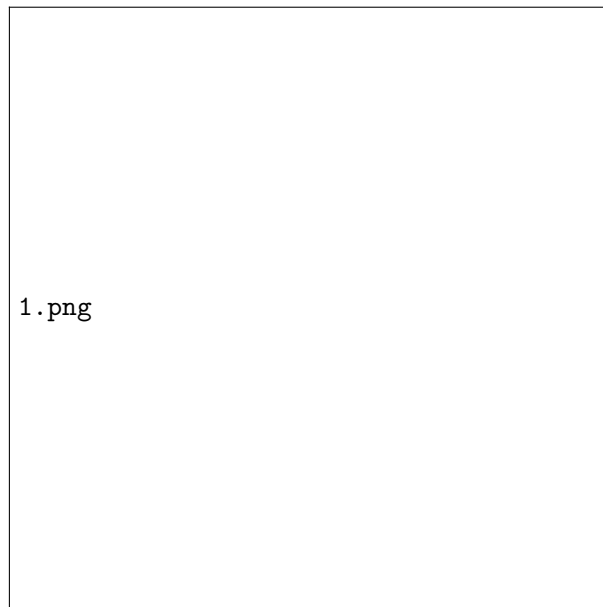


图 2: 各点坐标图

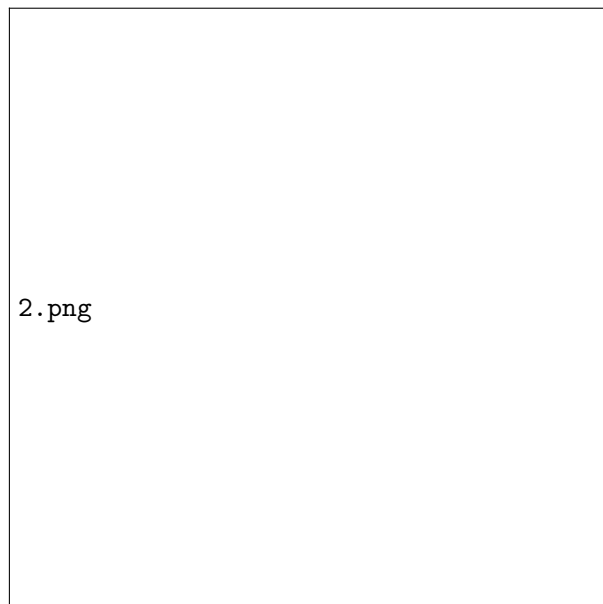


图 3: 仓库与目的地空间分布图I

题目要求忽略无人机的载重量，因此可以建立一条由一架无人机将所有包裹送到目的地的最短路径，这属于旅行商问题，可运用模拟退火算法求解。借助 Python 软件

编程，计算出一条最短路径，如图 4 和图 5 所示。

用于计算两个点之间的距离。对于两个点 (x_i, y_i) 和 (x_j, y_j) ，它们之间的距离为：

$$D[i][j] = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2} \quad (1)$$

用于计算整个路径的总距离。对于路径 path，总距离为：

$$\text{total_distance} = \sum_{i=0}^{n-1} D[\text{path}[i]][\text{path}[(i+1)\%n]] \quad (2)$$

其中， n 是路径中点的数量，由此可计算出路径的总距离，并在此基础上不断更新迭代找到最短路径。

用于决定是否接受新的路径。接受概率 P 为：

$$P = \begin{cases} \exp\left(-\frac{\Delta}{T}\right) & \text{if } \Delta \geq 0 \\ 1 & \text{if } \Delta < 0 \end{cases} \quad (3)$$

其中， Δ 是新路径与当前路径的距离差， T 是当前温度。在模拟退火算法的迭代过程中，不断更新确定是否接受新路径，当新路径的距离差 $\Delta = 10$ ，温度 $T = 100$ 时，计算得出接受概率 $P = \exp(-10/100) \approx 0.9048$ 。

$$\sum_{i=1}^n t_i \cdot x_{ij} \leq T \quad \forall j \in \{1, 2, \dots, n\} \quad (4)$$

每个节点的作业时间 $t_i = 5$ 分钟，无人机的最大飞行时间 $T = 300$ 分钟，对于路径中的每个节点 j ，计算得出总作业时间满足约束条件。

最终测得最短路径为 [13, 10, 14, 15, 16, 17, 21, 20, 19, 18, 11, 8, 3, 1, 2, 5, 4, 6, 7, 9, 12, 26, 27, 28, 29, 31, 30, 34, 36, 35, 33, 32, 25, 24, 23, 22, 13]，即从 13 号点出发飞往 10 号点，依此类推，最后返回 13 号点，从而实现将所有包裹送达所有目的地。最短路径总路程为 470.64。

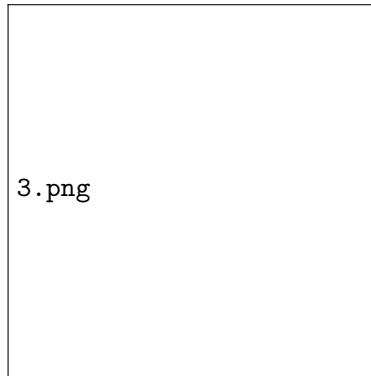


图 4: 运行模拟退火算法图

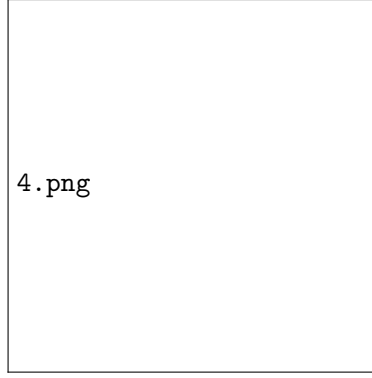


图 5: 最短路径图

5.2 问题二模型建立与求解

使用多架无人机递送包裹，在确保每架无人机装载量大致相等的前提下，目标是 minimized 所有无人机完成任务的总燃料消耗。

依据聚类算法（如 K-means）将目的地按地理位置分组，分组情况如表 2 所示。由于题目要求每架无人机装载量大致相同，对于 1 区、3 区、5 区，分别派遣两架无人机，2 区和 4 区分别派遣一架无人机。

然后对每组运用 TSP 算法中的启发式算法来规划路径。最终输出每架无人机的包裹分配列表以及每架无人机的飞行路径，可以简化为最小化所有无人机的总飞行距离，进而实现最小化总燃料消耗。最小化所有无人机的总飞行距离，以实现最小化总燃料消耗：

$$\min \sum_{i=1}^m \sum_{j=1}^{n_i} D[i][j] \quad (5)$$

其中， m 是无人机的数量， n_i 是第 i 架无人机访问的节点数量， $D[i][j]$ 是第 i 架无人机从节点 j 到下一个节点的距离。

每架无人机的装载量大致相等：

$$\left| \frac{k_i}{F} - \frac{k_j}{F} \right| \leq \epsilon \quad \forall i, j \quad (6)$$

其中， k_i 和 k_j 分别是第 i 和第 j 架无人机的装载量， F 是无人机的载重， ϵ 是一个很小的正数，表示允许的装载量差异。假设每架无人机的最大载重量 $F = 10$ 千克，允许的差异 $\epsilon = 0.1$ ，则每架无人机的装载量应在 9 千克到 11 千克之间。

将目的地按地理位置分组，每个子集由一架无人机服务。对于每个目的地 x_i ，计算其到各个聚类中心 c_j 的距离，并分配给最近的聚类中心：

$$x_i \in C_j \quad \text{if} \quad \arg \min_j \sqrt{(x_i - c_j)^2 + (y_i - c_j)^2} \quad (7)$$

其中， C_j 是第 j 个聚类。假设目的地 x_i 的坐标为 (17, 30)，聚类中心 c_1 的坐标为 (15,

15), 聚类中心 c_2 的坐标为 $(20, 20)$, 计算得出 x_i 到 c_1 的距离为:

$$\sqrt{(17-15)^2 + (30-15)^2} = \sqrt{4+225} = \sqrt{229} \approx 15.132 \text{ 千米}$$

到 c_2 的距离为:

$$\sqrt{(17-20)^2 + (30-20)^2} = \sqrt{9+100} = \sqrt{109} \approx 10.440 \text{ 千米}$$

因此, x_i 被分配到聚类 C_2 。

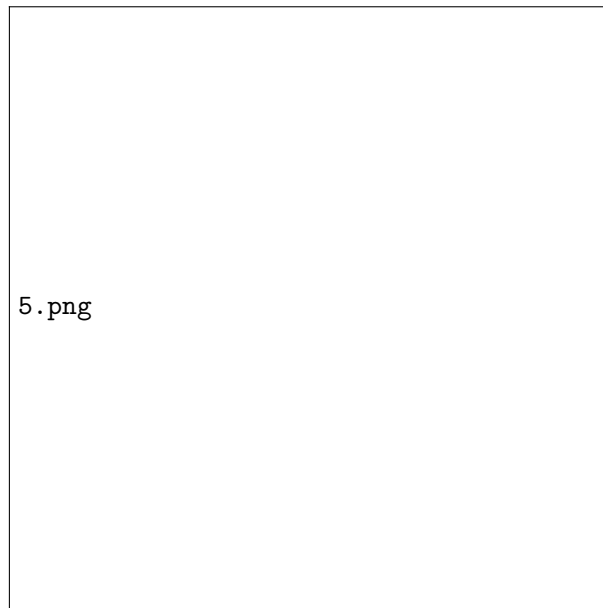
对于每个聚类后的子集, 求解TSP以规划无人机的路径。对于子集 S 中的节点, 寻找最短路径:

$$\min \sum_{i=1}^{|S|} D[S[i]][S[i+1]] \quad (8)$$

其中, $S[i]$ 表示路径中的第 i 个节点, 子集 S 包含节点 $A(52, 73)$ 、 $B(25, 34)$ 、 $C(21, 32)$, 计算得出路径 ABC 的总距离为 $\sqrt{(25-52)^2 + (34-73)^2} + \sqrt{(21-25)^2 + (32-34)^2} = \sqrt{2250} + \sqrt{20} \approx 47.434$ 千米。

通过上述模型和算法, 可以有效地为每架无人机分配包裹并规划飞行路径, 从而实现总燃料消耗的最小化。

表 2: 分区情况表



题目所给的无人机目的地共有 35 个, 因此定义无人机的载货量为 4 到 6 个。通过“需求量 / 载货量”计算, 大致需要 7 架无人机。以无人机出发点为中心, 将图随机划分为 7 个扇形区域, 每个扇形区域的角度约为 51° 。

将无人机的每个目的地视为一个点, 点密度 $P = \text{单个扇形区域的点的数量}$ (若点位于两个扇形区域的中间, 则将点划分到相对较少的扇形区域)。当扇形区域的点的数量小于 7 并大于等于 4 时, 将该区域定为疏区域; 当扇形区域的点的数量大于 7 并小

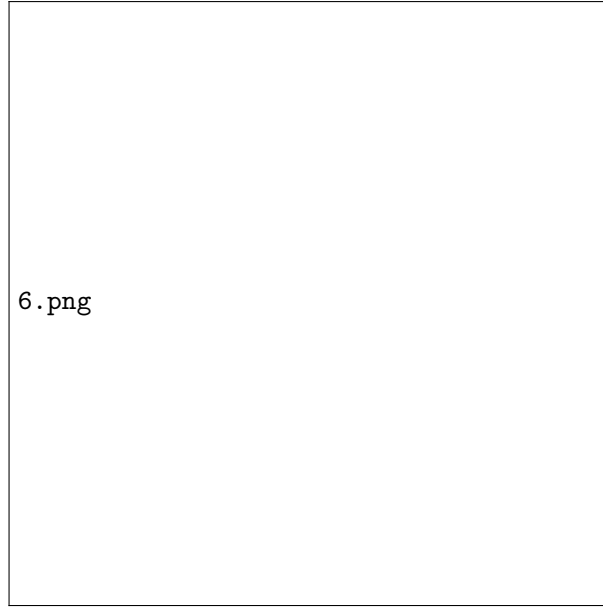


图 6: 仓库与目的地空间分布图II

于等于 12 时，将该区域定为密区域。由于点的空间分布不均，可以将一些不符合定义的区域进行拆分，然后合并到其他区域。

根据图中点的密度分布，将图划分为 5 个扇形区域，如图 6 所示。这种划分方式旨在更好地反映不同区域的需求点分布特征，为后续的路径规划提供更合理的分区基础。接着，针对每个扇形区域，我们分别运行模拟退火算法。在算法执行过程中，初始路径是随机生成的，然后通过不断地在当前路径的邻域内随机生成新的候选路径来更新迭代。对于每个候选路径，计算其对应的目标函数值，即路径长度，并与当前路径的目标函数值进行比较。得出以下结论：

(1)对于 1 区，由于目的地较多，属于密区域，因此派遣了两架无人机，路径分别为：

Path1 = 13 (52, 73), 2 (14, 24), 1 (16, 21), 3 (24, 20), 4 (23, 26), 13 (52, 73);

Path2 = 13 (52, 73), 7 (25, 34), 6 (21, 32), 5 (17, 30), 12 (36, 63), 13 (52, 73)。

最终测得 1 区最短路径距离为 248.31。

(2)对于 2 区，因为目的地较少，属于疏区域，可以视为旅行商问题 (TSP)，运用问题一的代码来获取最短路径，因此只需派遣一架无人机，路径如下：

Path = 13 (52, 73), 10 (50, 55), 9 (39, 47), 8 (40, 27), 11 (76, 24), 13 (52, 73)。

最终测得 2 区最短路径为 140.44。

(3)对于 3 区，由于目的地较多，属于密区域，因此派遣了两架无人机，路径分别为：

Path1 = 13 (52, 73), 17 (80, 70), 16 (77, 66), 15 (70, 67), 14 (63, 70),

13 (52, 73);

Path2 = 13 (52, 73), 18 (86, 67), 19 (85, 70), 20 (86, 75), 21 (81, 77), 13 (52, 73)。

最终测得 3 区最短路径为 136.70。

(4)对于 4 区，因为目的地较少，属于疏区域，可以视为旅行商问题 (TSP)，运用问题一的代码来获取最短路径，因此只需派遣一架无人机，路径如下：

Path = 13 (52, 73), 22 (52, 86), 23 (58, 98), 25 (69, 115), 24 (74, 109), 13 (52, 73)。

最终测得 4 区最短路径距离为 96.67。

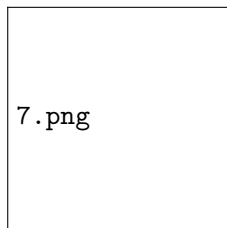
(5)对于 5 区，由于目的地较多，属于密区域，因此派遣了两架无人机，路径分别为：

Path1 = 13 (52, 73), 26 (37, 90), 31 (32, 106), 30 (28, 107), 29 (18, 99), 27 (24, 90), 13 (52, 73);

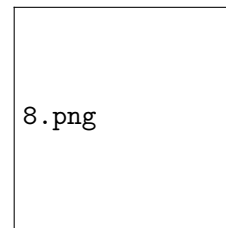
Path2 = 13 (52, 73), 28 (10, 84), 36 (13, 117), 35 (15, 119), 34 (18, 116), 33 (26, 120), 32 (36, 114), 13 (52, 73)。

最终测得 5 区最短路径距离为 248.18。

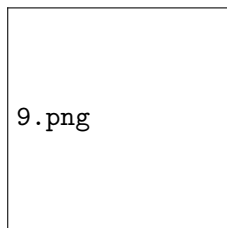
最后结果如图 7 所示：



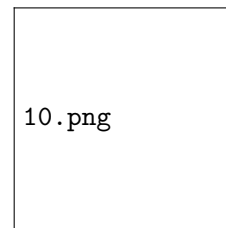
(a) 1 区最短路径图



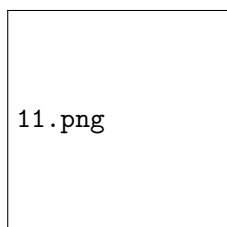
(b) 2 区最短路径图



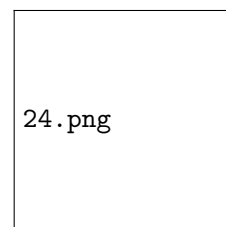
(c) 3 区最短路径图



(d) 4 区最短路径图



(e) 5 区最短路径图



(f) 总最短路径综合表

图 7：各区域最短路径图

5.3 问题三模型建立与求解

问题一 - 考虑单架无人机的载重量后对航程的规划

虽然需要考虑无人机的载重量对燃料消耗的影响，但本质上仍是一个旅行商问题。可采用动态规划或启发式算法，优先访问远距离、大载重的目的地。因此，建立数学模型如下：

无人机从起点出发时，所携带的包裹数量为 35，随后每经过一个目的地，所携带的包裹数量便会减少 1。当无人机经过最后一个目的地后，所携带的包裹数量变为 0。设无人机出发时所携带的包裹数量为 n ，每个包裹的重量都为 k ，无人机起点到第一个点的距离为 X_1 ，从第一个点到第二个点的距离为 X_2 ，以此类推，无人机经过的最后一个点到起点的距离为 X_{36} 。将无人机燃料的消耗关于载重定义为：

$$F = k [nX_1 + (n - 1)X_2 + (n - 2)X_3 + \dots + (n - 34)X_{35} + (n - 35)X_{36} + X_{36}] \quad (9)$$

当 F 值最小时，路径规划才最为合理。

引入新条件后，更新运用问题一所用的方法，如图 8 所示，即为最优路径。最优路径为：[13, 14, 15, 16, 17, 18, 19, 20, 21, 10, 8, 4, 3, 1, 2, 5, 6, 7, 9, 12, 22, 26, 30, 31, 23, 24, 25, 32, 33, 35, 36, 34, 29, 27, 28, 11, 13]，总距离为：576.79，其中 F 值为：7631.65。

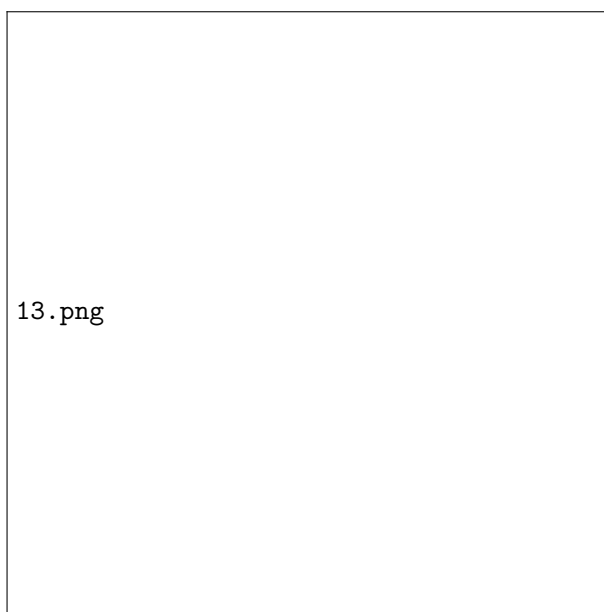


图 8：1区最优路径图

问题二 - 考虑多架无人机的载重量后对航程的规划

2 区和 4 区燃料消耗模型对于 2 区和 4 区，需要考虑无人机载重量对燃料消耗的影响。无人机从起点出发时，所携带的包裹数量为 4，随后每经过一个目的地，所携带的包裹数量便会减少 1。当无人机经过最后一个目的地后，所携带的包裹数量变为 0。设无人机每个包裹的重量都为 k ，无人机起点到第一个点的距离为 X_1 ，从第一个点到

第二个点的距离为 X_2 ，以此类推，无人机经过的最后一个点到起点的距离为 X_5 。将无人机燃料的消耗关于载重定义为：

$$F = k [4X_1 + 3X_2 + 2X_3 + 1X_4 + X_5] \quad (10)$$

当 F 值最小时，路径规划才最为合理。

通过 Python 软件编程，得到 2 区和 4 区的最优路径，如图 9 所示。2 区最优路径为：[13-10-9-8-11-13]，总距离为 142.42，其中 F 值为 243.98。4 区最优路径为：[13-22-23-25-24-13]，总距离为 96.67，其中 F 值为 182.75。

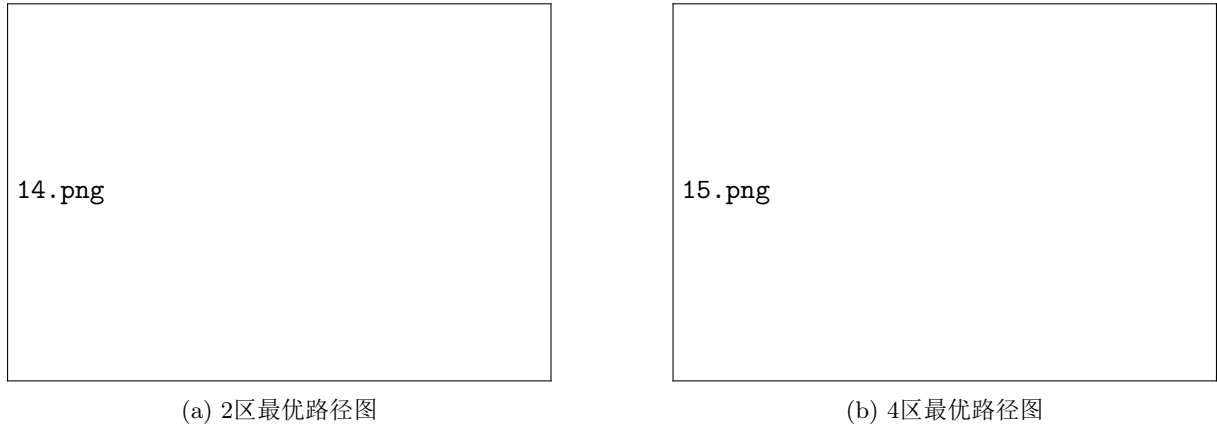


图 9：2 区和 4 区最优路径图

对于 1 区和 3 区，需要考虑无人机载重量对燃料消耗的影响。两架无人机从起点出发时，每架无人机所携带的包裹数量为 4，随后每经过一个目的地，所携带的包裹数量便会减少 1。当无人机所携带的包裹数量为 0 时，无人机直线返回起点。设无人机每个包裹的重量都为 k ，第一架无人机从起点到第一个点的距离为 X_1 ，从第一个点到第二个点的距离为 X_2 ，从第二个点到第三个点的距离为 X_3 ，从第三个点到第四个点的距离为 X_4 ，无人机经过的最后一个点到起点的距离为 X_5 ；第二架无人机从起点到第一个点的距离为 X_6 ，从第一个点到第二个点的距离为 X_7 ，从第二个点到第三个点的距离为 X_8 ，从第三个点到第四个点的距离为 X_9 ，无人机经过的最后一个点到起点的距离为 X_{10} 。

将第一架无人机燃料的消耗关于载重定义为：

$$F_1 = k [4X_1 + 3X_2 + 2X_3 + 1X_4 + X_5] \quad (11)$$

将第二架无人机燃料的消耗关于载重定义为：

$$F_2 = k [4X_6 + 3X_7 + 2X_8 + 1X_9 + X_{10}] \quad (12)$$

当 $F_1 + F_2$ 值最小时，路径规划才最为合理。通过 Python 软件编程，得到 1 区和 3 区的最优路径，如图 10 所示。

表3：1区的两条路径

路线	最优路径	距离	F值	总距离	总F值
1	13-6-5-2-1-13	129.43	299.29	253.58	550.33
2	13-12-7-4-3-13	124.15	251.04		

表4：3区的两条路径

路线	最优路径	距离	F值	总距离	总F值
1	13-14-15-16-17-13	59.25	115.76	136.70	296.90
2	13-21-20-19-18-13	77.45	181.14		

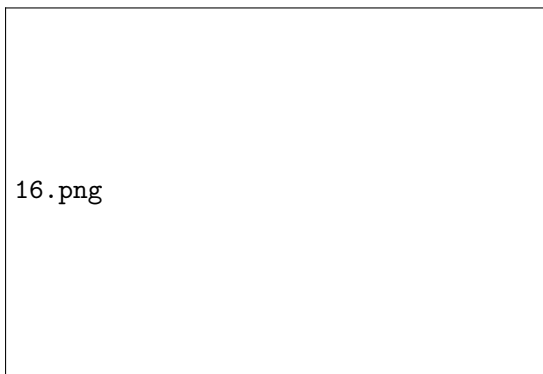
对于 5 区，需要考虑无人机载重量对燃料消耗的影响。两架无人机从起点出发，第一架无人机所携带的包裹数为 5，第二架无人机所携带的包裹数为 6，随后每架无人机每经过一个目的地，所携带的包裹数量便会减少 1。当无人机所携带的包裹数量为 0 时，无人机直线返回起点。设无人机每个包裹的重量都为 k ，第一架无人机从起点到第一个点的距离为 X_1 ，从第一个点到第二个点的距离为 X_2 ，从第二个点到第三个点的距离为 X_3 ，从第三个点到第四个点的距离为 X_4 ，从第四个点到第五个点的距离为 X_5 ，无人机经过的最后一个点到起点的距离为 X_6 ；第二架无人机从起点到第一个点的距离为 X_7 ，从第一个点到第二个点的距离为 X_8 ，从第二个点到第三个点的距离为 X_9 ，从第三个点到第四个点的距离为 X_{10} ，从第四个点到第五个点的距离为 X_{11} ，从第五个点到第六个点的距离为 X_{12} ，无人机经过的最后一个点到起点的距离为 X_{13} 。

根据动态规划的最优性原理，建立递推方程：

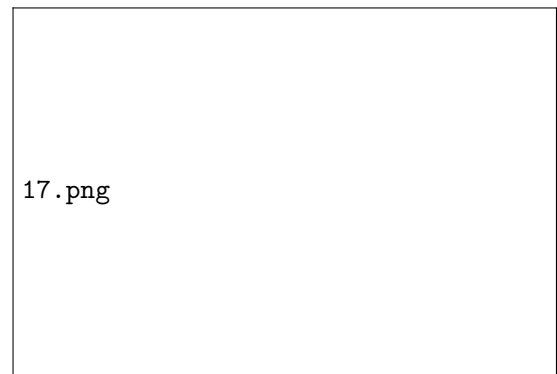
$$V_k(s_k) = \min_{u_k} [F_k(s_k, u_k) + V_{k+1}(s_{k+1})] \quad (13)$$

其中：

- $V_k(s_k)$ ：从阶段 k 状态 s_k 到最终阶段的最小燃料消耗。
- $V_{k+1}(s_{k+1})$ ：从阶段 $k+1$ 状态 s_{k+1} 到最终阶段的最小燃料消耗。



(a) 1区最优路径图



(b) 3区最优路径图

图 10：1区和 3 区最优路径图

当所有包裹都投递完毕时，无人机需要返回起点。边界条件为：

$$V_{N+1}(s_{N+1}) = k \times 0 \times X_{\text{pos}_{N+1}, \text{起点}} \quad (14)$$

将第一架无人机燃料的消耗关于载重定义为：

$$F_1 = k [5X_1 + 4X_2 + 3X_3 + 2X_4 + 1X_5 + X_6] \quad (15)$$

将第二架无人机燃料的消耗关于载重定义为：

$$F_2 = k [6X_7 + 5X_8 + 4X_9 + 3X_{10} + 2X_{11} + 1X_{12} + X_{13}] \quad (16)$$

当 $F_1 + F_2$ 值最小时，路径规划才最为合理。通过 Python 软件编程，得到 5 区的最优路径，如图 11 所示。

表5：5区的两条路径

路线	最优路径	距离	F值	总距离	总F值
1	13-26-27-29-33-32-13	124.63	298.43	264.42	699.33
2	13-31-30-34-35-36-28-13	139.79	400.90		

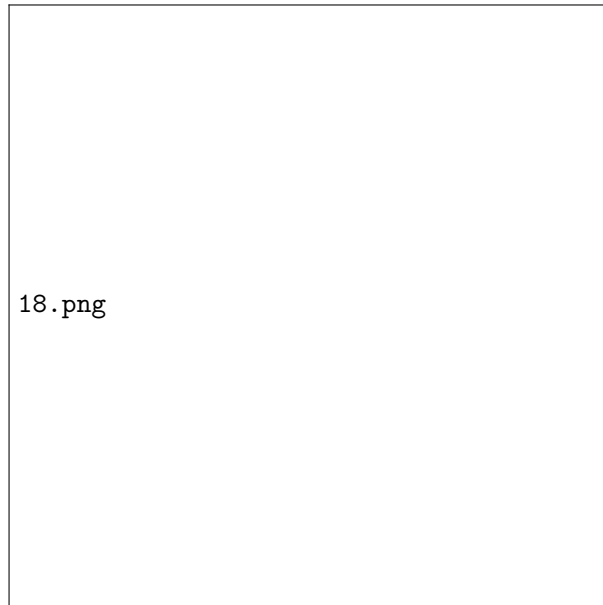


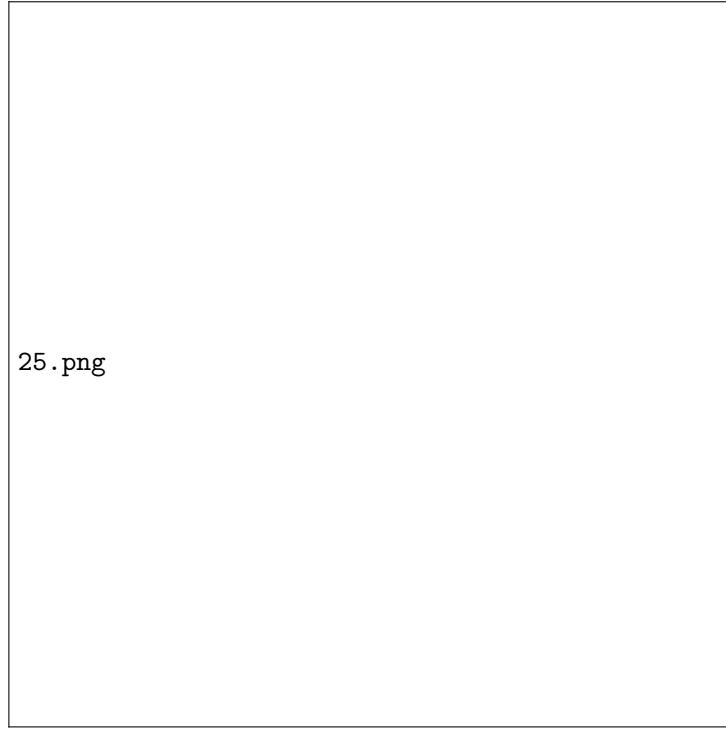
图 11：5区最优路径图

最终，对所有区域的路线进行汇总，如表 6，得出最终距离为893.79，总F值为1973.29.

5.4 问题四模型建立与求解

城市中存在障碍物约束的 TSP，路径需避开禁飞区。在考虑城市中存在禁飞区的情况下，重新对问题 1 进行建模，即安排单架无人机的目的地列表，使得无人机在避

表 6: 总路线表



开禁飞区的前提下，完成所有配送任务并返回出发点的总飞行距离最短。

障碍物成本函数

定义障碍物成本函数 $C_{\text{obstacle}}(\text{pos})$ ，表示无人机在位置 pos 时接近障碍物（禁飞区）的成本。可以用距离的倒数或二进制指示函数表示：

距离倒数形式：

$$C_{\text{obstacle}}(\text{pos}) = \frac{1}{d(\text{pos}, \text{obstacle})} \quad (17)$$

二进制指示形式：

$$C_{\text{obstacle}}(\text{pos}) = \begin{cases} 1 & \text{若 } \text{pos} \in \text{obstacle} \\ 0 & \text{否则} \end{cases} \quad (18)$$

阶段燃料消耗

考虑障碍物影响后，阶段 k 的燃料消耗公式为：

$$F_k(s_k, u_k) = k \times (n_k) \times X_{\text{pos}_k, \text{pos}_{k+1}} + \lambda \times C_{\text{obstacle}}(\text{pos}_{k+1}) \quad (19)$$

其中：

- λ ：障碍物成本权重，用于平衡燃料消耗与避开障碍物的优先级。
- $X_{\text{pos}_k, \text{pos}_{k+1}}$ ：从位置 pos_k 到位置 pos_{k+1} 的距离。

启发式函数：估计从当前节点到终点的最小距离（如欧几里得距离或曼哈顿距

离)。

$$h(n) = \text{distance}(n, \text{终点}) \quad (20)$$

在A*算法中，我们使用启发式函数 $h(n)$ 来估计从当前节点 n 到终点的最小距离，这可以帮助算法更高效地找到最优路径。具体来说，我们可以选择欧几里得距离（直线距离）或曼哈顿距离（网格距离）作为启发式函数。

路径成本函数：计算从起点到当前节点的实际成本。

$$g(n) = \sum_{i=1}^k \text{distance}(n_{i-1}, n_i) \quad (21)$$

路径成本函数 $g(n)$ 则计算从起点到当前节点 n 的实际飞行距离，它通过累加路径上每一段的距离来得到总成本。

总评估函数：综合实际成本和启发式估计，指导搜索方向。

$$f(n) = g(n) + h(n) \quad (22)$$

总评估函数 $f(n)$ 是启发式函数和路径成本函数的综合，用于指导搜索的方向，使得算法在探索过程中既能考虑实际成本，又能利用启发式信息快速逼近终点。

禁飞区约束：在搜索过程中，为了确保无人机避开禁飞区，我们在搜索过程中忽略所有位于禁飞区内的节点，这样生成的路径就不会穿过这些区域。

根据题目中给出的图 2 放入 Auto CAD 中，以图片左下角为坐标原点建立直角坐标系，测得两个四边形禁区顶点的坐标，然后使用 R 语言画出，如图 12 所示。

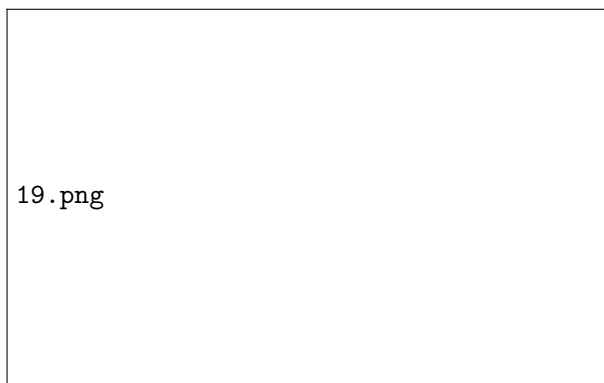


图 12: 带有禁飞区的空间分布图

方法一：由于禁飞区的阻挡，将无人机的目的地用一条直线分为上半部分和下半部分这两部分，如图 13 所示。

上半部分：求出无人机从仓库对应的点 13 出发，依次经过点 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 29, 30, 31, 32, 33, 34, 35, 36, 最后到达终点 28 且不经禁飞区的最短路径。这实际上是 TSP 问题的一个变种，通过 Python 软件编程，运用模拟退火算法（SA）计算出如图 14 所示的最短路径。

测得的最短路径为：13-14-15-16-17-18-19-20-21-22-23-24-25-32-33-35-36-34-30-31-26-

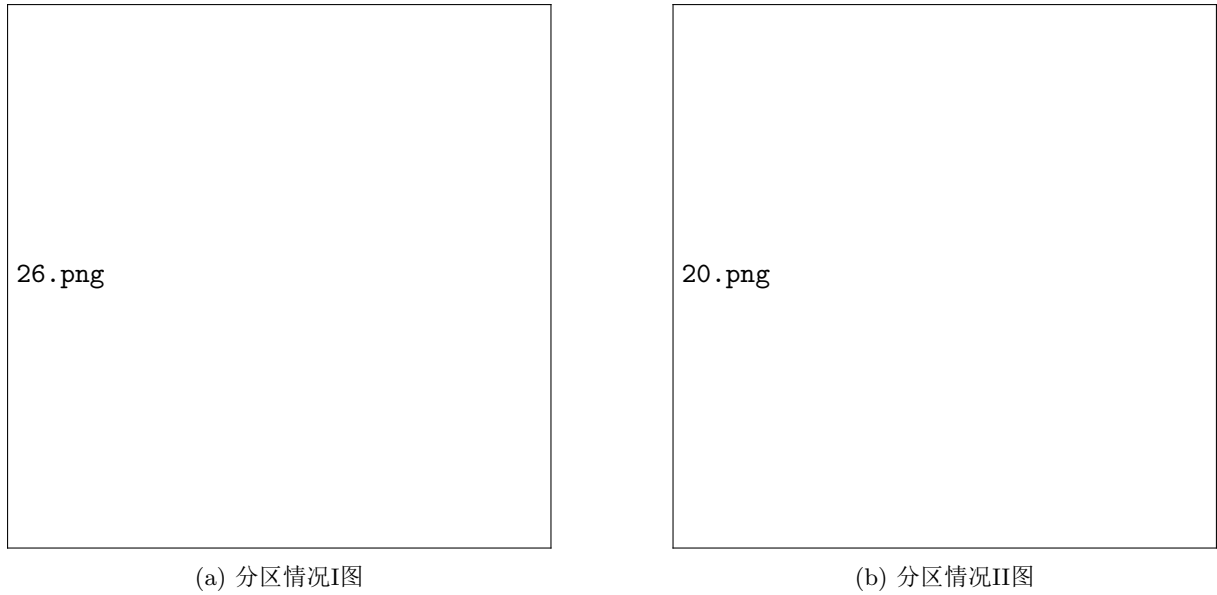


图 13: 上下两部分分区图

27-29-28，测定最短路径的距离为 261.26。

下半部分：求从点 28 出发，到达点 7 后，依次经过点 6，5，2，1，3，4，8，11，然后经过点 9，再经过点 10，12，最终到达点 13 且不经过禁飞区的最短路径。这也是 TSP 问题的一个变种，通过 Python 软件编程，运用模拟退火算法（SA）计算出如图 14 所示的最短路径。

测得的最短路径为：28-7-6-5-2-1-3-4-8-11-9-10-12-13，测定最短路径的距离为 230.92。

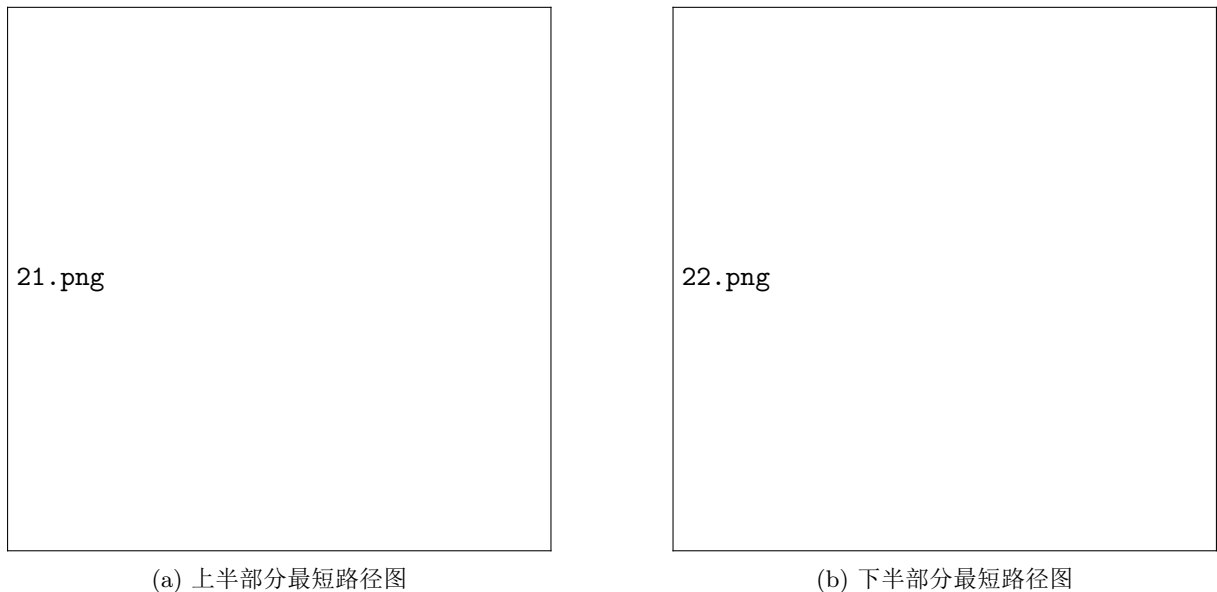


图 14: 上下两部分最短路径图

两部分加起来的最短路径为：

13-14-15-16-17-18-19-20-21-22-23-24-25-32-33-35-36-34-30-31-26-27-29-28-7-6-5-2-1-3-

4-8-11-9-10-12-13

最短总距离为：492.18

方法二：由于禁飞区的阻挡，无人机需要对禁飞区进行绕行，因此对第一题图 5 所示的最短路径图进行修改，如图 15 所示。

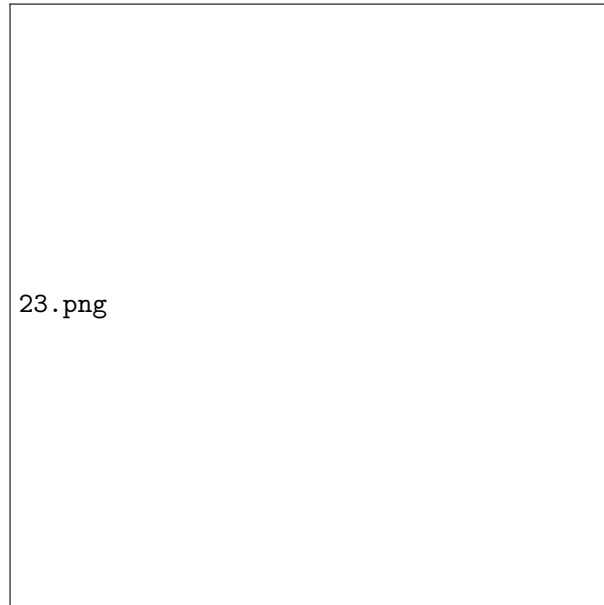


图 15：方法二最短路径图

方法二优化后的最短路径为 [13, 10, 14, 15, 16, 17, 21, 20, 19, 18, 禁飞区 2 顶点 2, 11, 8, 3, 1, 2, 5, 4, 6, 7, 9, 12, 禁飞区 1 顶点 2, 禁飞区 1 顶点 3, 26, 27, 28, 29, 31, 30, 34, 36, 35, 33, 32, 25, 24, 23, 22, 13]，测定最短路径的距离为 478.61。

表7：四边形禁飞区顶点坐标

	禁飞区1	禁飞区2
顶点1	(18, 63)	(59, 39)
顶点2	(44, 69)	(81, 36)
顶点3	(43, 87)	(70, 60)
顶点4	(22, 81)	(59, 57)

6 模型的评价

6.1 模型的优势

本研究的模型涵盖了从单架无人机到多架无人机的多种场景，并考虑了载重量对燃料消耗的影响以及禁飞区的限制。这些模型能够适应不同的实际需求，为无人机送

货规划提供了较为全面的解决方案。

通过引入模拟退火算法等路径搜索方法，模型能够在复杂的城市环境中为无人机规划出避开禁飞区的最优路径。同时，模型还考虑了无人机的载重量对燃料消耗的影响，使得规划结果更加贴近实际应用。

6.2 模型的局限性

在多架无人机的场景下，模型的求解复杂度较高，尤其是在考虑载重量对燃料消耗的影响时。这可能导致在大规模问题中求解时间较长，影响实际应用的效率。

假设条件：为了简化问题，模型中做出了一些假设，如无人机的飞行速度恒定、包裹的大小和重量差异忽略不计等。这些假设在一定程度上偏离了实际情况，可能会影响模型的准确性和适用性。

动态调整能力：模型主要针对静态的规划问题，对于动态变化的环境（如实时气象条件变化、临时的禁飞区调整等）的适应能力较弱。在实际应用中，需要进一步增强模型的动态调整能力，以应对复杂多变的实际情况。

7 模型推广

在城市物流配送方面，本模型可以直接应用于城市快递和外卖配送场景，通过合理规划无人机的航线和任务分配，提高配送效率，降低运营成本。

在应急救援方面，在自然灾害或紧急情况下，无人机可以快速将救援物资送达指定地点。本模型可以为无人机的路径规划和任务分配提供科学依据，提高应急救援的效率。

在农村和偏远地区物流方面，对于农村和偏远地区，道路条件较差，无人机送货可以作为一种有效的补充手段。本模型可以帮助规划最优的送货路径，确保物资能够及时送达。

8 参考文献与引用

参考文献

- [1] 吕伟,吴嗣贤,吴子涵,等.基于时间窗的无人机配送应急物资路径规划研究[J].中国安全生产科学技术,2025,21(03):170-176.
- [2] 张慧敏,王葆宗,陆东生,等.邮政物流中无人机路径规划优化研究[J].邮政研究,2025,41(02):3-9.DOI:10.13955/j.yzyj.2025.02.01.07.
- [3] 李从庆.基于模拟退火算法的车辆取送货运输路径研究[J].郑州铁路职业技术学院学报,2025,37(01):49-53.DOI:10.13920/j.cnki.zztlyzjsxyxb.2025.01.027.
- [4] 胡悦,孙智达,黄惠.面向无人机路径规划的可视分析系统[J/OL].图学学报,1-12[2025-04-13].<http://kns.cnki.net/kcms/detail/10.1034.T.20250224.1028.002.html>.

附录

问题一原始运行代码

```

1      import numpy as np
2      import matplotlib.pyplot as plt
3      import random
4      import math
5
6      # 定义城市坐标
7      cities = np.array([
8          [16, 21],      # City 1
9          [14, 24],      # City 2
10         [24, 20],      # City 3
11         [23, 26],      # City 4
12         [17, 30],      # City 5
13         [21, 32],      # City 6
14         [25, 34],      # City 7
15         [40, 27],      # City 8
16         [39, 47],      # City 9
17         [50, 55],      # City 10
18         [76, 24],      # City 11
19         [36, 63],      # City 12
20         [52, 73],      # City 13
21         [63, 70],      # City 14
22         [70, 67],      # City 15
23         [77, 66],      # City 16
24         [80, 70],      # City 17
25         [86, 67],      # City 18
26         [85, 70],      # City 19
27         [86, 75],      # City 20
28         [81, 77],      # City 21
29         [52, 86],      # City 22
30         [58, 98],      # City 23
31         [74, 109],     # City 24
32         [69, 115],     # City 25
33         [37, 90],      # City 26
34         [24, 90],      # City 27
35         [10, 84],      # City 28
36         [18, 99],      # City 29
37         [28, 107],     # City 30
38         [32, 106],     # City 31
39         [36, 114],     # City 32
40         [26, 120],     # City 33
41         [18, 116],     # City 34
42         [15, 119],     # City 35
43         [13, 117],     # City 36
44     ])
45
46     # Check the number of coordinates

```

```

47 if len(cities) != 36:
48     raise ValueError("The number of cities must be 36")
49
50 # Display city coordinates
51 print("City coordinates:")
52 for i, (x, y) in enumerate(cities):
53     print(f"City {i+1}: ({x:.2f}, {y:.2f})")
54
55 # Calculate the distance between two cities
56 def distance(city1, city2):
57     return np.sqrt((city1[0] - city2[0])**2 + (city1[1] - city2[1])**2)
58
59 # Calculate the total distance of a path
60 def total_distance(path):
61     total = 0.0
62     for i in range(len(path)):
63         total += distance(cities[path[i]], cities[path[(i+1)%len(path)]])
64     return total
65
66 # Simulated Annealing Algorithm
67 def simulated_annealing(cities, initial_temp, cooling_rate, num_iterations):
68     num_cities = len(cities)
69
70     # Initial path: random permutation
71     current_path = list(range(num_cities))
72     random.shuffle(current_path)
73
74     current_distance = total_distance(current_path)
75     best_path = current_path.copy()
76     best_distance = current_distance
77
78     temp = initial_temp
79
80     for i in range(num_iterations):
81         # Generate a new path: randomly swap two cities
82         new_path = current_path.copy()
83         a, b = random.sample(range(num_cities), 2)
84         new_path[a], new_path[b] = new_path[b], new_path[a]
85
86         new_distance = total_distance(new_path)
87
88         # Calculate the energy difference
89         delta = new_distance - current_distance
90
91         # Decide whether to accept the new path
92         if delta < 0 or random.random() < math.exp(-delta / temp):
93             current_path = new_path
94             current_distance = new_distance
95
96         # Update the best path

```

```

97         if current_distance < best_distance:
98             best_path = current_path.copy()
99             best_distance = current_distance
100
101     # Lower the temperature
102     temp *= cooling_rate
103
104     # Print progress
105     if i % 10000 == 0:
106         print(f"Iteration {i}: Current Distance = {current_distance:.2f},
107               Best Distance = {best_distance:.2f}, Temperature = {temp:.2f}")
108
109     return best_path, best_distance
110
111 # Run the Simulated Annealing Algorithm
112 initial_temp = 10000
113 cooling_rate = 0.9995
114 num_iterations = 100000
115
116 print("\nStarting Simulated Annealing Algorithm...")
117 best_path, best_distance = simulated_annealing(cities, initial_temp,
118                                               cooling_rate, num_iterations)
119
120 print("\nOptimal Path Order (City Indices):", best_path)
121 print("Optimal Path Length:", best_distance)
122
123 # Visualize the results
124 def plot_path(cities, path, title):
125     plt.figure(figsize=(12, 8))
126
127     # Plot cities
128     plt.scatter(cities[:, 0], cities[:, 1], c='red', s=100)
129     for i, (x, y) in enumerate(cities):
130         plt.text(x, y, str(i+1), color='black', ha='center', va='center',
131                 fontsize=8)
132
133     # Plot the path
134     for i in range(len(path)):
135         start = cities[path[i]]
136         end = cities[path[(i+1)%len(path)]]
137         plt.plot([start[0], end[0]], [start[1], end[1]], 'b-', linewidth=1)
138
139     plt.title(title, fontsize=14)
140     plt.xlabel('X Coordinate', fontsize=12)
141     plt.ylabel('Y Coordinate', fontsize=12)
142     plt.grid(True)
143     plt.show()
144
145 # Plot the optimal path
146 plot_path(cities, best_path, f"Optimized Path using Simulated Annealing

```

```

144     Algorithm (Distance: {best_distance:.2f})")
145 # Print the city visiting order
146 print("\nCity Visiting Order (Starting from City 13):")
147 for i, city_idx in enumerate(best_path):
148     print(f"Step {i+1}: Visit City {city_idx+1} (Coordinates: {cities[city_idx]
        [0]:.2f}, {cities[city_idx][1]:.2f})")

```

问题二原始运行代码

```

1  import itertools
2  import math
3  import matplotlib.pyplot as plt
4
5  def calculate_distance(p1, p2):
6      return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)
7
8  def solve_tsp(points):
9      n = len(points)
10     if n == 0:
11         return [], 0
12     # Generate all possible paths and compute the shortest
13     shortest_path = None
14     min_distance = float('inf')
15     for perm in itertools.permutations(points):
16         current_distance = 0
17         # From start to first point
18         current_distance += calculate_distance([52, 73], perm[0])
19         # Between points in the permutation
20         for i in range(n - 1):
21             current_distance += calculate_distance(perm[i], perm[i+1])
22         # Back to start
23         current_distance += calculate_distance(perm[-1], [52, 73])
24         if current_distance < min_distance:
25             min_distance = current_distance
26             shortest_path = list(perm)
27     return shortest_path, min_distance
28
29 def find_optimal_paths(points):
30     n = len(points)
31     # Generate all possible combinations of 5 points for the first person
32     best_total_distance = float('inf')
33     best_path1 = []
34     best_path2 = []
35     for combo in itertools.combinations(points, 5):
36         set1 = list(combo)
37         set2 = [p for p in points if p not in set1]
38         path1, dist1 = solve_tsp(set1)
39         path2, dist2 = solve_tsp(set2)
40         total_distance = dist1 + dist2

```

```

41         if total_distance < best_total_distance:
42             best_total_distance = total_distance
43             best_path1 = path1
44             best_path2 = path2
45     return best_path1, best_path2, best_total_distance
46
47 def plot_paths(path1, path2):
48     plt.figure(figsize=(10, 8))
49     # Plot all points
50     points = [ ] # Enter coordinates here
51     for point in points:
52         plt.plot(point[0], point[1], 'bo')
53         plt.text(point[0], point[1], f'({point[0]},{point[1]})')
54     # Start and end point
55     start = [52, 73]
56     plt.plot(start[0], start[1], 'ro')
57     plt.text(start[0], start[1], 'Start/End (52,73)')
58     # Plot path1
59     if path1:
60         path1_x = [start[0]] + [p[0] for p in path1] + [start[0]]
61         path1_y = [start[1]] + [p[1] for p in path1] + [start[1]]
62         plt.plot(path1_x, path1_y, 'r-', label='Person 1 Path')
63     # Plot path2
64     if path2:
65         path2_x = [start[0]] + [p[0] for p in path2] + [start[0]]
66         path2_y = [start[1]] + [p[1] for p in path2] + [start[1]]
67         plt.plot(path2_x, path2_y, 'g-', label='Person 2 Path')
68     plt.xlabel('X Coordinate')
69     plt.ylabel('Y Coordinate')
70     plt.title('Optimal Paths for Two Persons')
71     plt.legend()
72     plt.grid()
73     plt.show()
74
75 def main():
76     points = [ ] # Enter coordinates here
77     path1, path2, total_distance = find_optimal_paths(points)
78     print("Optimal Path for Person 1 (4 points):", path1)
79     print("Optimal Path for Person 2 (5 points):", path2)
80     print("Total shortest distance:", total_distance)
81     plot_paths(path1, path2)
82
83 if __name__ == "__main__":
84     main()

```

问题三中对问题一的原始运行代码

```

1     import numpy as np
2     import matplotlib.pyplot as plt
3     from math import sqrt

```

```

4 import random
5
6 # Define the start point and the list of points to visit
7 start = [52, 73]
8 points = [
9     [16, 21], [14, 24], [24, 20], [23, 26],
10    [17, 30], [21, 32], [25, 34], [36, 63]
11 ]
12
13 # Function to compute the Euclidean distance between two points
14 def distance(p1, p2):
15     return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)
16
17 # Function to compute the objective function F for a given route
18 def calculate_F(permutation):
19     X = []
20     # From start to the first point
21     X.append(distance(start, permutation[0]))
22     # Between intermediate points
23     for i in range(len(permutation) - 1):
24         X.append(distance(permutation[i], permutation[i+1]))
25     # From last point back to start
26     X.append(distance(permutation[-1], start))
27     # Calculate F value
28     F = 4 * X[0] + 3 * X[1] + 2 * X[2] + X[3] + X[4]
29     total_distance = sum(X)
30     return F, total_distance, X
31
32 # Initialization of variables to store the optimal solution
33 min_total_F = float('inf')
34 best_route1 = None
35 best_route2 = None
36 best_F1 = 0
37 best_F2 = 0
38 best_distance1 = 0
39 best_distance2 = 0
40 best_X1 = []
41 best_X2 = []
42
43 # Iterate over all combinations of 4 points for the first plane
44 for combo in itertools.combinations(points, 4):
45     remaining = [p for p in points if p not in combo]
46     # Iterate over all permutations of the selected 4 points for the first plane
47     for perm1 in itertools.permutations(combo):
48         F1, distance1, X1 = calculate_F(perm1)
49         # Iterate over all permutations of the remaining 4 points for the second plane
50         for perm2 in itertools.permutations(remaining):
51             F2, distance2, X2 = calculate_F(perm2)
52             total_F = F1 + F2
53             if total_F < min_total_F:

```

```

54         min_total_F = total_F
55         best_route1 = perm1
56         best_route2 = perm2
57         best_F1 = F1
58         best_F2 = F2
59         best_distance1 = distance1
60         best_distance2 = distance2
61         best_X1 = X1
62         best_X2 = X2
63
64     # Output the results
65     print("Optimal Routes:")
66     print("Plane 1 Route: Start ->", " -> ".join([str(p) for p in best_route1]), "
        -> Start")
67     print("Plane 2 Route: Start ->", " -> ".join([str(p) for p in best_route2]), "
        -> Start")
68     print("Plane 1 Flight Distance:", best_distance1)
69     print("Plane 2 Flight Distance:", best_distance2)
70     print("Plane 1 F Value:", best_F1)
71     print("Plane 2 F Value:", best_F2)
72     print("Total F (F1 + F2):", min_total_F)
73     print("Plane 1 Segment Distances:", best_X1)
74     print("Plane 2 Segment Distances:", best_X2)
75
76     # Plotting the optimal routes
77     plt.figure(figsize=(10, 8))
78
79     # Plot the start point
80     plt.scatter(start[0], start[1], color='red', label='Start', s=100)
81     plt.text(start[0], start[1], 'Start', fontsize=12, ha='right')
82
83     # Plot the intermediate points
84     for point in points:
85         plt.scatter(point[0], point[1], color='blue')
86         plt.text(point[0], point[1], str(point), fontsize=10, ha='right')
87
88     # Plot Plane 1's route
89     route1 = [start] + list(best_route1) + [start]
90     x1 = [p[0] for p in route1]
91     y1 = [p[1] for p in route1]
92     plt.plot(x1, y1, color='green', label='Plane 1', marker='o')
93
94     # Plot Plane 2's route
95     route2 = [start] + list(best_route2) + [start]
96     x2 = [p[0] for p in route2]
97     y2 = [p[1] for p in route2]
98     plt.plot(x2, y2, color='purple', label='Plane 2', marker='o')
99
100     plt.xlabel('X')
101     plt.ylabel('Y')

```

```

102 plt.title('Optimal Flight Paths')
103 plt.legend()
104 plt.grid(True)
105 plt.show()

```

问题三中对问题二(5区)的原始运行代码

```

1     import itertools
2     import math
3     import matplotlib.pyplot as plt
4     import random
5
6     # Define points
7     start = [52, 73]
8     points = [
9         [37, 90], [24, 90], [10, 84], [18, 99], [28, 107],
10        [32, 106], [36, 114], [26, 120], [18, 116], [15, 119], [13, 117]
11    ]
12
13    # Calculate distance between two points
14    def distance(a, b):
15        return math.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)
16
17    # Calculate F1 and D1
18    def calculate_F1_D1(path):
19        X = []
20        X.append(distance(start, path[0]))
21        for i in range(len(path)-1):
22            X.append(distance(path[i], path[i+1]))
23        X.append(distance(path[-1], start))
24        weights = [5, 4, 3, 2, 1, 1]
25        F1 = sum(w * x for w, x in zip(weights, X))
26        D1 = sum(X)
27        return F1, D1, X
28
29    # Calculate F2 and D2
30    def calculate_F2_D2(path):
31        X = []
32        X.append(distance(start, path[0]))
33        for i in range(len(path)-1):
34            X.append(distance(path[i], path[i+1]))
35        X.append(distance(path[-1], start))
36        weights = [6, 5, 4, 3, 2, 1, 1]
37        F2 = sum(w * x for w, x in zip(weights, X))
38        D2 = sum(X)
39        return F2, D2, X
40
41    # Greedy algorithm to generate path order (to reduce number of permutations)
42    def greedy_path(start, points, is_plane1=True):
43        unvisited = points.copy()

```

```

44     path = []
45     current = start
46     while unvisited:
47         # Sort by distance
48         unvisited.sort(key=lambda p: distance(current, p))
49         # Select next point based on weights in objective function
50         next_p = unvisited[0]
51         path.append(next_p)
52         current = next_p
53         unvisited.remove(next_p)
54     return path
55
56 # Generate all possible allocations
57 min_total = float('inf')
58 best_F1 = None
59 best_D1 = None
60 best_F2 = None
61 best_D2 = None
62 best_path1 = None
63 best_path2 = None
64
65 # Iterate through all C(11,5) combinations
66 for selected_indices in itertools.combinations(range(len(points)), 5):
67     # Points for the first plane
68     path1_points = [points[i] for i in selected_indices]
69     # Points for the second plane
70     path2_points = [points[i] for i in range(len(points)) if i not in
71                     selected_indices]
72
73     # Try all permutations for path1 (since 5! = 120 is manageable)
74     for perm1 in itertools.permutations(path1_points):
75         F1, D1, X1 = calculate_F1_D1(perm1)
76         # For path2, use greedy algorithm to generate initial path
77         perm2 = greedy_path(start, path2_points, is_plane1=False)
78         F2, D2, X2 = calculate_F2_D2(perm2)
79         total = F1 + F2
80         if total < min_total:
81             min_total = total
82             best_F1, best_D1 = F1, D1
83             best_F2, best_D2 = F2, D2
84             best_path1 = perm1
85             best_path2 = perm2
86             print(f"New min: {min_total}")
87
88 # Optionally, try random sampling for some permutations (optional)
89 for _ in range(20):
90     perm2 = random.sample(path2_points, len(path2_points))
91     F2, D2, X2 = calculate_F2_D2(perm2)
92     total = F1 + F2
93     if total < min_total:

```

```

93         min_total = total
94         best_F1, best_D1 = F1, D1
95         best_F2, best_D2 = F2, D2
96         best_path1 = perm1
97         best_path2 = perm2
98         print(f"New min: {min_total}")
99
100 # Output the results
101 print("\nOptimal Solution Found:")
102 print("Best F1 + F2:", min_total)
103 print("Best F1:", best_F1)
104 print("Best D1:", best_D1)
105 print("Best path1:", best_path1)
106 print("Best F2:", best_F2)
107 print("Best D2:", best_D2)
108 print("Best path2:", best_path2)
109
110 # Plot the routes
111 def plot_path(start, path, color, label):
112     x = [start[0]] + [p[0] for p in path] + [start[0]]
113     y = [start[1]] + [p[1] for p in path] + [start[1]]
114     plt.plot(x, y, 'o-', color=color, label=label)
115     for i, p in enumerate(path):
116         plt.text(p[0], p[1], str(i+1))
117
118 plt.figure(figsize=(12, 8))
119 plot_path(start, best_path1, 'blue', 'Plane 1')
120 plot_path(start, best_path2, 'red', 'Plane 2')
121 plt.scatter(start[0], start[1], color='green', s=100, label='Start', zorder=5)
122 plt.legend()
123 plt.xlabel('X Coordinate')
124 plt.ylabel('Y Coordinate')
125 plt.title('Optimal Flight Paths for Two Planes')
126 plt.grid()
127 plt.show()

```

问题四的原始运行代码

```

1 import heapq
2
3 # Define the map and forbidden zones
4 def create_map():
5     # Here should contain your map creation logic
6     # Return a dictionary where keys are nodes and values are lists of adjacent nodes
7     return {}
8
9 def is_forbidden(node, forbidden_zones):
10     # Determine if the node is within a forbidden zone
11     for zone in forbidden_zones:
12         if node in zone:

```

```

13         return True
14     return False
15
16 # A* algorithm implementation
17 def a_star(start, goal, graph, forbidden_zones):
18     open_set = []
19     heapq.heappush(open_set, (0, 0, start, [start]))
20
21     # Set of visited nodes
22     visited = set()
23
24     while open_set:
25         current_f, current_g, current_node, path = heapq.heappop(open_set)
26
27         # Check if the goal node is reached
28         if current_node == goal:
29             return path, current_g # Return the path and total distance
30
31         # Skip if the current node has been visited
32         if current_node in visited:
33             continue
34         visited.add(current_node)
35
36         # Iterate over neighboring nodes
37         for neighbor in graph[current_node]:
38             # Skip nodes within forbidden zones
39             if is_forbidden(neighbor, forbidden_zones):
40                 continue
41
42             # Calculate path cost
43             cost = current_g + distance(current_node, neighbor)
44
45             # Heuristic function to estimate remaining distance
46             h = distance(neighbor, goal)
47             f = cost + h
48
49             # Add the neighbor node to the priority queue
50             if neighbor not in visited:
51                 heapq.heappush(open_set, (f, cost, neighbor, path + [neighbor])
52
53             # If the queue is empty and no path is found, return an empty path and infinite distance
54             return [], float('inf')
55
56 # Calculate the distance between two nodes (using Euclidean distance)
57 def distance(node1, node2):
58     # Assume nodes are tuples (x, y)
59     return ((node1[0] - node2[0])**2 + (node1[1] - node2[1])**2) ** 0.5
60
61 # Example usage

```

```

62 if __name__ == "__main__":
63     # Create the map
64     graph = create_map()
65     # Define forbidden zones (example: two forbidden zones, each is a set of nodes)
66     forbidden_zones = [
67         { (1, 2), (1, 3), (2, 3), (2, 2) },
68         { (5, 5), (5, 6), (6, 6), (6, 5) }
69     ]
70     # Define start and goal nodes
71     start = (0, 0)
72     goal = (7, 7)
73     # Run the A* algorithm
74     path, total_distance = a_star(start, goal, graph, forbidden_zones)
75
76     if path:
77         print(f"Path found: {path}")
78         print(f"Total distance: {total_distance}")
79     else:
80         print("No path found")

```